NORTHSTAR TECHNOLOGIES PVT. LTD.

ENGINEERING STANDARDS

Engineering team guidelines for software development, quality, security, and delivery

Document ID	ENG-GDE-001
Document Type	GUIDE
Access Scope	ENGINEERING_ONLY
Department	Engineering
Version	1.0
Effective Date	04 November 2026
Document Owner	Engineering Department
Review Cycle	Every 6 months or after significant engineering changes

Fictional demonstration document for the OnBoardIQ environment. The tools, review thresholds, and workflow examples are illustrative and should be adapted to approved engineering processes before operational use. This guide is intended for the Engineering department and is not a company-wide policy.

1. Purpose and scope

This guide establishes consistent engineering practices for designing, building, reviewing, testing, securing, deploying, and maintaining software at Northstar Technologies Pvt. Ltd. It is intended for employees in the Engineering department, including software engineers, QA engineers, engineering leads, and engineering managers. Other departments should follow their own approved operational standards.

2. Engineering principles

- **Maintainability:** Prefer clear, modular, well-documented solutions over unnecessary complexity.
- **Reliability:** Design for failure, validate inputs, handle errors, and monitor important services.
- **Security by design:** Protect data and systems throughout development, testing, and deployment.
- **Quality ownership:** Every engineer is responsible for the quality of the code they contribute.
- **Collaboration:** Share context, review constructively, and document decisions that affect others.
- **Continuous improvement:** Learn from incidents, feedback, and delivery outcomes.

3. Repository and source-control standards

All production code and engineering documentation must be stored in company-approved version-control repositories. Use individual accounts and multi-factor authentication where supported.

- Create branches from the team's agreed base branch. Use descriptive names such as feature/user-profile, fix/payment-timeout, or chore/update-dependencies.
- Keep commits focused and use clear messages that explain the change, for example: 'Add pagination to employee directory'.
- Do not commit credentials, API keys, private keys, access tokens, personal data, build artifacts, or local environment files.
- Use approved secret-management solutions and environment-specific configuration.
- Protect main or release branches with required reviews and automated checks.
- Resolve merge conflicts carefully and avoid force-pushing shared branches unless the team workflow explicitly permits it.

4. Coding standards and maintainability

Code should be readable, consistent with the language and framework conventions used by the project, and easy for another engineer to understand. Teams should use automated formatters, linters, and static-analysis tools where practical.

- Use meaningful names for variables, functions, classes, modules, and API fields.
- Keep functions and modules focused; separate business logic, data access, presentation, and infrastructure concerns where appropriate.
- Avoid duplicated logic when a well-defined reusable component or function is appropriate.
- Validate external inputs and handle expected errors explicitly; do not expose stack traces or sensitive internal details to end users.
- Add comments for non-obvious decisions and complex business rules, not for code that is already self-explanatory.
- Remove dead code, temporary debugging statements, and unused dependencies before merging.
- Follow the project's agreed naming, formatting, folder-structure, and dependency-management conventions.

5. API and database design

- Design APIs around clear resource and action semantics. Use consistent HTTP methods, status codes, pagination, and error response formats.
- Validate request parameters and payloads at service boundaries. Apply authentication and authorization on every protected operation.

- Document API contracts and material changes so dependent teams can coordinate safely.
- Use database constraints, appropriate indexes, and transactions to preserve data integrity.
- Avoid exposing internal database identifiers or sensitive fields unless required and authorized.
- Use migration scripts for schema changes; review them for compatibility, rollback considerations, and potential data loss.
- Do not run destructive queries or migrations against production without the required review and approval.

6. Code review and pull-request process

All non-trivial changes to protected branches must be submitted through a pull request or the team's approved review workflow. The author should provide enough context for reviewers to understand the purpose, implementation, testing, and risks.

- Keep pull requests focused and reasonably sized. Split unrelated changes into separate requests.
- Include a summary, linked issue or task, relevant screenshots or API examples where useful, and test evidence.
- At least one qualified engineer who did not author the change should review it; critical or high-risk changes may require additional approval.
- Review for correctness, readability, security, performance, test coverage, and backward compatibility.
- Address review comments respectfully and resolve all required discussions before merging.
- Do not approve changes you have not reviewed or bypass mandatory checks without documented authorization.

7. Testing and quality assurance

Testing must be appropriate to the risk and impact of the change. Engineers should add or update tests as part of the implementation rather than treating testing as a final separate activity.

- **Unit tests:** Verify individual functions, components, and business rules.
- **Integration tests:** Verify interactions between services, databases, queues, and external dependencies.
- **API and contract tests:** Verify request validation, response formats, authentication, authorization, and compatibility.
- **End-to-end tests:** Cover important user journeys and critical workflows.
- **Regression tests:** Confirm that existing behaviour remains correct after changes.
- Test failure cases, boundary values, invalid input, permission restrictions, and relevant performance conditions.
- Do not use production personal or confidential data in test environments unless explicitly approved and protected.

A change should not be merged when required automated checks fail, unless an authorized exception is documented with a clear risk assessment and follow-up action.

8. Application security requirements

- Follow secure coding practices and the company's IT Security Policy (IT-POL-001).
- Enforce authentication and server-side authorization; never rely only on frontend visibility or client-supplied role claims.
- Protect against common application risks such as injection, cross-site scripting, cross-site request forgery, insecure direct object references, and unsafe deserialization.
- Use secure password handling and approved authentication libraries; never store plaintext passwords.
- Encrypt sensitive data in transit and use approved encryption and key-management mechanisms for data at rest where required.
- Keep dependencies current, review vulnerability alerts, and remove packages that are no longer needed.
- Log security-relevant events without logging passwords, tokens, secrets, or unnecessary personal data.

9. Architecture and technical documentation

Significant design decisions should be recorded in an architecture decision record (ADR) or an equivalent engineering document. Documentation should explain the problem, chosen approach, alternatives considered, trade-offs, and consequences.

- Maintain service diagrams, data-flow diagrams, API documentation, and operational notes for systems where they are useful.
- Document service ownership, dependencies, configuration requirements, and critical failure modes.
- Review major changes to service boundaries, databases, messaging, authentication, or infrastructure with relevant technical owners.
- Prefer supported, consistent platform patterns unless a documented requirement justifies a different approach.

10. Environments, configuration, and secrets

- Keep development, testing, staging, and production environments logically separated.
- Use environment variables or approved configuration services for non-secret configuration.
- Store secrets only in approved secret managers; restrict access and rotate them when exposure is suspected.
- Use least-privilege service accounts and scoped credentials for service-to-service communication.
- Do not copy production databases or customer data into local environments without explicit approval and required safeguards.

11. Build, deployment, and release management

Production releases must follow the team's approved CI/CD and change-management process. A release should be traceable to reviewed source changes and validated build artifacts.

- Run required automated tests, code-quality checks, and security checks in the pipeline.
- Use reviewed and versioned deployment configuration. Avoid undocumented manual production changes.
- Communicate release scope, expected impact, dependencies, and maintenance windows where applicable.
- Prepare a rollback or recovery approach for changes that could affect service availability or data integrity.
- Verify service health after deployment and monitor relevant logs, metrics, and alerts.
- Record release versions and significant production changes in the approved system.

12. Reliability, observability, and incident response

Services should expose enough operational information to help teams understand availability, errors, latency, and resource usage. Use approved logging, metrics, tracing, dashboards, and alerting systems appropriate to the service.

- Use structured logs and meaningful error messages while avoiding secrets and unnecessary personal information.
- Define health checks and alerts for critical services and dependencies.
- Assign clear service ownership and maintain operational runbooks for important systems.
- During an incident, prioritize user and data safety, communicate through the incident process, and preserve relevant evidence.
- Report suspected security incidents immediately to IT & Information Security; do not wait for a routine engineering review.
- After significant incidents, participate in blameless reviews that identify contributing factors and actionable prevention work.

13. Performance and scalability

Engineers should consider expected load, resource usage, and growth when designing services. Measure before optimizing, and use representative tests for performance-sensitive changes.

- Use pagination, bounded queries, caching, and asynchronous processing where appropriate.
- Set timeouts and sensible retry policies for network calls; avoid uncontrolled retry loops.
- Protect services from unbounded requests and resource exhaustion through appropriate limits and rate controls.
- Review database query plans and indexes for high-volume or latency-sensitive paths.
- Document known capacity assumptions and performance constraints for critical services.

14. Dependency and open-source management

Use approved package registries and libraries. Before introducing a new dependency, consider maintenance activity, licensing, security history, compatibility, and whether the functionality is already available in the project. Follow company processes for license review and software procurement where applicable.

15. Engineering collaboration and knowledge sharing

- Participate constructively in planning, estimation, design discussions, reviews, and retrospectives.
- Keep task status and blockers visible in the team's approved project-management tool.
- Share technical knowledge through documentation, demos, pairing, and mentoring.
- Provide respectful, specific, actionable feedback in code reviews and technical discussions.
- Escalate delivery risks early and coordinate changes that affect other teams or services.

16. Exceptions and deviations

A deviation from these standards must have a documented business or technical reason, an identified owner, an assessment of security and operational risk, and approval from the Engineering Lead or designated engineering authority. Security-related exceptions must also be reviewed by IT & Information Security. Exceptions should be time-limited where possible and tracked to closure.

17. Roles and responsibilities

Role	Responsibilities
Software Engineer	Build, test, document, and maintain software; follow review, security, and release practices.
QA Engineer	Define and execute test coverage; report defects and quality risks; validate releases.
Engineering Lead	Set team conventions, review technical designs, approve appropriate exceptions, and support quality.
Engineering Manager	Ensure delivery processes, ownership, staffing, and standards adoption.
IT & Information Security	Provide security requirements, review security exceptions, and support incident response.

18. Review and document administration

The Engineering Department owns this guide and will review it at least every six months or after significant changes to the engineering toolchain, architecture, security requirements, or delivery process. The latest approved version will be published in the company document repository. Questions should be directed to the Engineering Lead or engineering-standards@northstar.example. For security concerns, contact security@northstar.example.

Employee acknowledgement

I acknowledge that I have received and read the Northstar Technologies Pvt. Ltd. Engineering Standards (ENG-GDE-001, Version 1.0). I understand that these standards apply to my Engineering team responsibilities and that I should seek approval before deviating from required practices.

Employee name: _____

Team / role: _____

Signature: _____ Date: _____

End of document • ENG-GDE-001 • Northstar Technologies Pvt. Ltd.